A.D. PATEL INSTITUTE OF TECHNOLOGY
(The Charutar Vidya Mandal University)
DEPARTMENT OF INFORMATION TECHNOLOGY

MYSTERYBOX

A Python-Based Detective Game

MINI PROJECT REPORT

Information Technology

Submitted by: Megha Suthar(12402080601082)

Vira Patel(12402080601151)

Class:IT-B

Faculty Guide:Prof. Kavya Prajapati

Academic Year: 2026–27

A. D. Patel Institute of Technology

New Vallabh Vidyanagar, Anand, Gujarat

ABSTRACT

MysteryBox is a desktop-based detective game developed using Python and Tkinter. The player selects a mystery case and difficulty level, investigates locations, questions suspects, collects clues, and makes a final accusation. If the accusation is correct, the player answers a short quiz related to the case. The game awards points for finding clues, recording statements, identifying the culprit, and answering quiz questions. A speed bonus is also available for unused actions.

The project separates its code into three main parts: data, logic, and GUI. Case stories and clue information are stored in separate Python files. The Game class handles actions, clue dependencies, score calculation, hints, accusations, and final results. Tkinter is used to display the main window, buttons, status indicators, investigation log, notebook, and dialogue boxes. A JSON file is used to store high scores.

The project provides a practical way to learn functions, dictionaries, lists, loops, conditions, modules, event-driven programming, and basic file handling. The code was checked for Python syntax and compilation. Full interactive testing should still be performed on the target computer before formal submission.

1. INTRODUCTION

1.1 Background

Python is a high-level programming language that is widely used for learning programming, automation, data processing, and application development. It includes standard libraries that make it possible to build small desktop applications without installing many additional packages. Tkinter is Python's standard interface to the Tk GUI toolkit and can be used to create windows, buttons, labels, menus, and dialogue boxes.

MysteryBox applies these concepts to a detective-game scenario. Instead of only displaying text, the application responds to the player's choices. Each choice may reveal evidence, record a statement, change the score, or unlock another part of the investigation.

1.2 Problem Statement

A beginner programming project should demonstrate more than isolated statements or a command-line calculation. The aim is to develop a small interactive application where the user can make choices and observe the effect of those choices. MysteryBox addresses this by presenting a case-based investigation with rules, evidence dependencies, scoring, and a graphical interface.

1.3 Objectives

- Develop a desktop game using Python and Tkinter.
- Organise the project into separate data, logic, and GUI modules.
- Allow users to investigate locations and question suspects.
- Make some clues available only after prerequisite evidence is collected.
- Implement difficulty levels, action limits, hints, scoring, and ranks.
- Ask final questions after a correct accusation.
- Store high scores in a JSON file.
- Practise readable, modular, and maintainable Python code.

1.4 Scope

The current project is a single-user, offline desktop application. It includes four mystery scenarios: The Missing Blue Diamond, The Stolen Painting, The Leaked Exam Paper, and The Night Train Necklace. It does not use a database, online account, network service, or external game engine.

2. REQUIREMENTS AND TECHNOLOGIES

2.1 Functional Requirements

- The application should open a main game window.
- The player should be able to select a case and a difficulty level.
- The game should show the case story, locations, and suspects.
- Searching a location should reveal a clue when its prerequisites are satisfied.
- Questioning a suspect should reveal the next available statement.
- The game should track score, remaining actions, clues, and hints.
- The player should be able to request a hint and make an accusation.
- A correct accusation should start a final quiz; a wrong accusation should end the case.
- The application should display the final score and rank.
- High scores should be saved and loaded from a JSON file.

2.2 Non-Functional Requirements

- Usability: controls should be labelled clearly and grouped by purpose.
- Maintainability: case data, rules, and GUI should be in separate modules.
- Portability: the application should run on a system with compatible Python and Tkinter.
- Reliability: the program should handle missing or unreadable high-score data without crashing during loading.
- Performance: the cases are small and should respond quickly on a normal desktop.

2.3 Hardware and Software

Item	Requirement
Computer	Desktop or laptop
Operating system	Windows, macOS, or Linux with a supported Tkinter installation
Language	Python 3
GUI library	Tkinter and ttk
Data storage	JSON text file for high scores
Editor (optional)	Visual Studio Code, IDLE, or another Python editor

2.4 Libraries Used

The project uses standard Python libraries. tkinter builds the interface; random selects and shuffles final quiz questions and options; json reads and writes high scores; os creates the score-file path; and tkinter.messagebox/simpledialog provide standard dialogue boxes and input prompts.

3. SYSTEM DESIGN

3.1 Architecture Overview

MysteryBox uses a simple three-part structure. The data layer contains the case stories, suspects, clues, quiz questions, and difficulty settings. The logic layer manages the state of a game and applies the rules. The GUI layer displays the game and sends user actions to the logic layer.

Folder / File	Responsibility
main.py	Creates the Tkinter root window and starts the application.
data/__init__.py	Combines all case definitions and difficulty settings.
data/diamond.py	Defines The Missing Blue Diamond case.
data/painting.py	Defines The Stolen Painting case.
data/exam.py	Defines The Leaked Exam Paper case.
data/train.py	Defines The Night Train Necklace case.
data/difficulty.py	Defines action, hint, penalty, quiz, bonus, and rank settings.
logic/game.py	Implements investigation rules, scoring, hints, accusations, quiz, and ranks.
logic/highscores.py	Loads and saves the high-score list in JSON format.
gui/app.py	Builds the main window, controls, investigation log, notebook, and event handlers.
gui/dialogs.py	Contains pop-up dialogue functions.
gui/theme.py	Stores colours, fonts, and a reusable coloured button class.

3.2 Data Flow

The user clicks a location or suspect button in the GUI. The GUI calls the corresponding method in the Game class. The logic checks the current case data and the clues or statements already collected. It updates the score, action count, notebook, and game status, then returns messages. The GUI displays those messages in the investigation log and refreshes the status bar.

3.3 Main Data Structures

Dictionaries are used for cases, locations, suspects, clue definitions, and difficulty settings because each item has a named key. Lists store clues found, statements heard, notebook entries, and quiz questions selected for a game. Tuples are used for log messages that contain a message category and its text.

4. GAME WORKFLOW AND ALGORITHMS

4.1 Game Workflow

1. Start the program and create the main window.
2. Select one of the four cases and choose Easy, Medium, or Hard.
3. Read the story and the short descriptions of the suspects.
4. Search locations or question suspects. Each action uses one turn.
5. Read the investigation log and check the detective notebook.
6. Use a hint if needed; a hint reduces the score and hint count.
7. Make an accusation when enough evidence has been collected.
8. If the accusation is wrong, the case ends with a penalty.
9. If the accusation is correct, answer the selected final questions.
10. Calculate the final score and rank, display the result, and optionally save the score.

4.2 Clue Dependency Algorithm

Clues are not always available immediately. Each clue has a list of prerequisite IDs. The program checks whether every prerequisite is present in the collected clues or heard statements. If the requirements are met, the next clue for that location can be found. This creates a simple investigation chain.

Pseudocode:

1. Receive the selected location.
2. Read the clues defined for that location.
3. For each clue, check whether it has already been found.
4. Check whether all prerequisite IDs are known.
5. If a clue is available, add it to the notebook and award points.
6. Otherwise, display a message that no new evidence is available.
7. Reduce the action count and refresh the interface.

4.3 Suspect Questioning

Each suspect has an ordered list of statements. A statement may require a clue or an earlier statement. When the player questions a suspect, the logic finds the next statement whose requirements are satisfied, records it, awards statement points, and adds any newly unlocked clue.

4.4 Scoring

Event	Score effect
Find a new clue	+10 points
Hear a new suspect statement	+5 points
Correct accusation	+50 points
Correct final quiz answer	+10 points
Use a hint	−5 points

Wrong accusation	Penalty depends on difficulty
Finish with unused actions	Speed bonus depends on difficulty

5. GRAPHICAL USER INTERFACE

5.1 Main Window

The main window is created in gui/app.py. It contains the MysteryBox heading, a New Game button, case selection, difficulty controls, a status bar, action buttons, an investigation log, and a detective notebook. The layout is designed to keep the player's actions and collected evidence visible while the game is running.

5.2 Interface Components

Component	Purpose
Case selector	Chooses which mystery story to play.
Difficulty controls	Selects action, hint, penalty, quiz, bonus, and rank settings.
Location buttons	Search places connected to the current case.
Suspect buttons	Question a suspect and reveal available statements.
Status bar	Displays score, actions left, clues found, hints left, and level.
Investigation log	Shows story details, clues, statements, and feedback.
Detective notebook	Keeps a readable record of collected evidence.
Get Hint	Suggests a useful next action at a score cost.
Make Accusation	Opens a dialogue to choose the suspected culprit.
Game menu	Provides New Game, High Scores, and Exit.
Help menu	Provides How to Play and About information.

5.3 Theme and Dialogue Boxes

The gui/theme.py module stores the colours and fonts used by the application and defines a reusable ColorButton. The gui/dialogs.py module keeps several pop-up interactions separate from the main window code. This reduces repetition and makes the GUI file easier to follow.

5.4 Usability Considerations

The score and action count are visible during play. Clues and statements are shown in different text styles, while warning and success messages use separate colours. The player can start another case using New Game and can view saved scores from the Game menu.

6. IMPLEMENTATION DETAILS

6.1 Application Entry Point

The main.py file imports tkinter and MysteryBoxApp, creates the root window, constructs the application object, and starts the Tkinter event loop with root.mainloop(). This keeps the entry point short and makes the overall startup sequence easy to understand.

6.2 Game Class

The Game class in logic/game.py stores the selected case and level. It initialises the score, action count, available hints, clues found, statements heard, notebook entries, and final quiz questions. Its methods include search(), question(), use_hint(), accuse(), check_answer(), finish(), rank(), and summary_lines().

6.3 Difficulty Settings

The data/difficulty.py file defines Easy, Medium, and Hard. The settings specify the number of actions, number of final quiz questions, available hints, wrong-accusation penalty, wasted-move penalty, speed-bonus multiplier, and score thresholds for ranks. The current configuration gives each level 20 actions, while other settings differ.

6.4 Case Data

Each case file defines a dictionary named CASE. It includes the title, story, culprit ID, locations, suspects, clue definitions, and final questions. The data/__init__.py file imports the four cases and exposes them through a single CASES dictionary. This structure makes it possible to add or modify a case without changing the core GUI.

6.5 High Scores

The logic/highscores.py module uses JSON to store a list of scores in highscores.json in the project folder. When a score is saved, the entries are sorted from highest to lowest and only the top ten are retained. If the file does not exist or cannot be read, loading returns an empty list.

6.6 Event-Driven Programming

Tkinter works through events. A button is connected to a function using its command parameter. When the player clicks the button, the related function runs, calls the game logic, writes the returned messages to the log, and refreshes the display. The event loop keeps the interface responsive while waiting for user input.

7. TESTING AND REVIEW

7.1 Review Performed

The supplied ZIP was extracted and its Python source files were reviewed. All 14 Python files passed Python syntax parsing and compilation checks. The four case definitions were checked for their main data counts and for prerequisite IDs that refer to missing clue or statement IDs. No missing prerequisite IDs were found in that static check.

7.2 Case Inventory

Case	Locations	Suspects	Clue entries	Quiz questions
The Missing Blue Diamond	5	4	11	10
The Stolen Painting	5	5	13	12
The Leaked Exam Paper	5	5	12	10
The Night Train Necklace	5	5	12	10

7.3 Test Cases

Note: the table above describes functional test cases for execution. Static syntax and data checks were completed on the supplied source; the full GUI test cases should be run manually on the target computer before claiming that all tests passed.

8. RESULTS, LIMITATIONS AND FUTURE SCOPE

8.1 Result

The reviewed source code is organised into data, logic, and GUI packages. The project defines four mystery cases and provides a common game engine to process them. The code compiles successfully and its clue/statement prerequisite references pass a static consistency check. The GUI is implemented with Tkinter and includes the investigation controls, status bar, log, notebook, hints, accusation flow, final quiz, and high-score menu.

8.2 Current Limitations

- The application is designed for one local player and does not include online multiplayer.
- Case stories and clue data are stored directly in Python dictionaries rather than an external editor or database.
- High scores are stored locally in a JSON file and are not synchronised between computers.
- The game's investigation paths are predefined; it does not use artificial intelligence to generate new cases.
- Full interactive GUI testing depends on running the application in a desktop environment with Tkinter.

8.3 Future Enhancements

- Add more mystery cases and optional case packs.
- Add automated unit tests for clue dependencies, scoring, and game-ending conditions.
- Add save/resume support for an unfinished case.
- Improve keyboard navigation and accessibility.
- Add a case editor that stores stories in JSON files.
- Add a results screen with score history and simple statistics.
- Package the program for easier installation while keeping the source code available.

8.4 Learning Outcomes

The project provides practice in modular programming, dictionaries and lists, condition checking, random selection, event handling, GUI construction, and JSON file operations. It also shows how a program can separate its data from its rules and presentation.

9. CONCLUSION

MysteryBox is a Python-based detective game created to demonstrate programming fundamentals through an interactive desktop application. The player investigates one of four cases by searching locations, questioning suspects, collecting clues, using hints, and making an accusation. Difficulty settings change the challenge, and a final quiz rewards the player for understanding the evidence.

The project's modular structure is a key feature. The data package stores case information, the logic package applies the game rules and handles high scores, and the GUI package manages the window and user interaction. The source-code review confirmed that all 14 Python files compile and that the case prerequisite references are internally consistent in the static checks performed. Manual testing of the complete GUI remains necessary on the intended computer.

Overall, MysteryBox is suitable as a student-level mini project because its features can be explained using core Python concepts without requiring a database or third-party framework. It can be extended in the future with additional cases, automated tests, and improved save features.

REFERENCES

- Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
- Python Software Foundation. tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
- Python Software Foundation. json — JSON encoder and decoder. https://docs.python.org/3/library/json.html